

School of Computer Science and Artificial Intelligence

Lab Assignment 11.2

Program	: B. Tech (CSE)
Specialization	: -
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	:II
Academic Session	: 2025-2026
Name of Student	: M.Srinath
Enrollment No.	: 2403A51L29
Batch No.	51
Date	: 13/03/26

Task-1:

Prompt:Create a Python class implementing a Stack data structure.

Requirements:

- Use a list internally.
- Methods: push(), pop(), peek(), is_empty(), size().
- Follow LIFO behavior.
- Handle pop from empty stack safely.
- Add docstrings, comments, and a small example showing stack operations.

Code:

```

1 class Stack:
2     """
3     A simple Stack (LIFO) data structure implementation using a Python list.
4     """
5     def __init__(self):
6         """
7         Initialize an empty stack.
8         """
9         # Internal list to store stack elements
10        self._items = []
11
12    def push(self, item):
13        """
14        Push an item onto the top of the stack.
15        """
16        #param item: The item to be added to the stack.
17        """
18        self._items.append(item)
19
20    def pop(self):
21        """
22        Remove and return the top item of the stack.
23        """
24        # Handles popping from an empty stack safely by returning None.
25        """
26        #return: The top item if the stack is not empty, otherwise None.
27        """
28        if self.is_empty():
29            # Safely handle empty stack: return None instead of raising an error
30            return None
31        return self._items.pop()
32
33    def peek(self):
34        """
35        Return the top item of the stack without removing it.
36        """
37        #return: The top item if the stack is not empty, otherwise None.
38        """
39        if self.is_empty():
40            return None
41        return self._items[-1]
42
43    def is_empty(self):
44        """
45        Check whether the stack is empty.
46        """
47        #return: True if the stack has no element, False otherwise.
48        """
49        return len(self._items) == 0
50
51    def size(self):
52        """
53        Return the number of items currently in the stack.
54        """
55        #return: Integer size of the stack.
56        """
57        return len(self._items)
58
59
60 if __name__ == "__main__":
61     # Small example showing stack operations
62
63     stack = Stack()
64
65     print("Is stack empty?", stack.is_empty()) # True
66
67     # Push some items
68     stack.push(10)
69     stack.push(20)
70     stack.push(30)
71
72     print("Current size:", stack.size()) # 3
73     print("Top element (peek):", stack.peek()) # 30
74
75     # Pop elements
76     print("Popped:", stack.pop()) # 30
77     print("Popped:", stack.pop()) # 20
78
79     print("Current size:", stack.size()) # 1
80     print("Top element (peek):", stack.peek()) # 10
81
82     # Pop remaining element
83     print("Popped:", stack.pop()) # 10
84
85     # Safely pop from empty stack
86     print("Popped from empty stack:", stack.pop()) # None
87     print("Is stack empty now?", stack.is_empty()) # True
88

```

Output:

```
Is stack empty? True
Current size: 3
Top element (peek): 30
Popped: 30
Popped: 20
Current size: 1
Top element (peek): 10
Popped: 10
Popped from empty stack: None
Is stack empty now? True
```

Task -2:

Prompt: Write a Python program implementing a Queue class.

Requirements:

- Follow FIFO principle.
- Methods: enqueue(), dequeue(), front(), is_empty(), size().
- Handle empty queue cases.
- Add comments, docstrings, and a short demo of queue

operations. Code:

```
1 class Queue:
2     """
3     A simple FIFO (First-In-First-Out) queue implementation.
4     """
5
6     def __init__(self):
7         """Initialize an empty queue."""
8         self._items = []
9
10    def is_empty(self):
11        """
12        Check if the queue is empty.
13
14        Returns:
15            bool: True if the queue has no elements, False otherwise.
16        """
17        return len(self._items) == 0
18
19    def enqueue(self, item):
20        """
21        Add an item to the back of the queue.
22
23        Args:
24            item: The element to be added.
25        """
26        # Insert at index 0, so the "front" is at the end of the list.
27        self._items.insert(0, item)
28
29    def dequeue(self):
30        """
31        Remove and return the item from the front of the queue.
32
33        Returns:
34            The item at the front of the queue.
35
36        Raises:
37            IndexError: If the queue is empty.
38        """
39        if self.is_empty():
40            raise IndexError("dequeue from empty queue")
41        # Pop from the end, which represents the front of the queue.
42        return self._items.pop()
43
44    def front(self):
45        """
46        Return (without removing) the item at the front of the queue.
47
48        Returns:
49            The item at the front of the queue.
50
51        Raises:
52            IndexError: If the queue is empty.
53        """
54        if self.is_empty():
55            raise IndexError("front from empty queue")
56        return self._items[-1]
57
58    def size(self):
59        """
60        Return the number of items in the queue.
61
62        Returns:
63            int: Number of elements currently in the queue.
64        """
65        return len(self._items)
```

```

67
68 def demo_queue_operations():
69     """
70     Demonstrate basic operations of the Queue class.
71     """
72     q = Queue()
73
74     print("Is queue empty?", q.is_empty())
75     print("Enqueueing 10, 20, 30...")
76     q.enqueue(10)
77     q.enqueue(20)
78     q.enqueue(30)
79
80     print("Current size:", q.size())
81     print("Front element:", q.front()) # Should be 10 (first in)
82
83     print("Dequeued:", q.dequeue()) # Removes 10
84     print("Dequeued:", q.dequeue()) # Removes 20
85     print("Current front:", q.front()) # Should be 30
86     print("Current size:", q.size())
87     print("Is queue empty?", q.is_empty())
88
89     # Demonstrate handling of empty queue
90     print("\nClearing remaining elements...")
91     q.dequeue() # remove 30
92
93     print("Is queue empty now?", q.is_empty())
94     try:
95         q.dequeue()
96     except IndexError as e:
97         print("Error when dequeuing from empty queue:", e)
98
99     try:
100         q.front()
101     except IndexError as e:
102         print("Error when getting front from empty queue:", e)
103
104
105 if __name__ == "__main__":
106     demo_queue_operations()
107
108

```

Output:

```

Is queue empty? True
Enqueueing 10, 20, 30...
Is queue empty? True
Enqueueing 10, 20, 30...
Enqueueing 10, 20, 30...
Current size: 3
Front element: 10
Dequeued: 10
Current size: 3
Front element: 10
Dequeued: 10
Dequeued: 10
Dequeued: 20
Current front: 30
Dequeued: 20
Current front: 30
Current front: 30
Current size: 1
Is queue empty? False
Current size: 1
Is queue empty? False

Is queue empty? False

Clearing remaining elements...
Is queue empty now? True
Error when dequeuing from empty queue: dequeue from empty queue
Error when getting front from empty queue: front from empty queue
Error when getting front from empty queue: front from empty queue

```

Task-3:

Prompt: Create a Python implementation of a Singly Linked List.

Requirements:

- Node class with data and next pointer.
- LinkedList class with insert_at_beginning(), insert_at_end(), and display().
- Show traversal of the list.
- Include comments, docstrings, and a simple usage example.

Code:

```

1  """
2
3  This module defines:
4  - Node: a single node in a singly linked list.
5  - LinkedList: a singly linked list with basic insertion and display operations.
6
7  It also contains a simple usage example that demonstrates:
8  - Inserting nodes at the beginning and at the end.
9  - Traversing and displaying the list contents.
10 """
11
12 from __future__ import annotations
13 from typing import Any, Optional
14
15 class Node:
16     """
17     Represents a single node in a singly linked list.
18
19     Attributes
20     -----
21     data : Any
22         The value stored in the node.
23     next : Optional[Node]
24         Reference to the next node in the list, or None if this is the last node.
25     """
26
27     def __init__(self, data: Any) -> None:
28         """
29         Initialize a new node with the given data.
30
31         Parameters
32         -----
33         data : Any
34             The value to store in the node.
35         """
36         self.data = data
37         self.next = Optional[Node] = None
38
39     def __repr__(self) -> str:
40         """Return a string representation of the node."""
41         return f"Node({self.data})"
42
43
44 class LinkedList:
45     """
46     A simple singly linked list implementation.
47
48     The list maintains a reference to the head node. New nodes can be
49     inserted at the beginning or at the end, and the list can be traversed
50     to display all elements.
51     """
52
53     def __init__(self) -> None:
54         """Initialize an empty linked list."""
55         self.head = Optional[Node] = None
56
57     def insert_at_beginning(self, data: Any) -> None:
58         """
59         Insert a new node containing 'data' at the beginning of the list.
60
61         This operation runs in O(1) time.
62         """
63         new_node = Node(data)
64         new_node.next = self.head
65         self.head = new_node

```

```

66
67     def insert_at_end(self, data: Any) -> None:
68         """
69         Insert a new node containing 'data' at the end of the list.
70
71         This operation runs in O(n) time because it traverses the list
72         to find the last node.
73         """
74         new_node = Node(data)
75
76         # If the list is empty, the new node becomes the head.
77         if self.head is None:
78             self.head = new_node
79             return
80
81         # Otherwise, traverse to the last node.
82         current = self.head
83         while current.next is not None:
84             current = current.next
85
86         # Link the last node to the new node.
87         current.next = new_node
88
89     def display(self) -> None:
90         """
91         Traverse the list and print each node's data.
92
93         This demonstrates list traversal from head to tail.
94         """
95         current = self.head
96         elements = []
97
98         # Traverse the list, collecting values.
99         while current is not None:
100             elements.append(str(current.data))
101             current = current.next
102
103         # Join collected values with arrows to visualize the links.
104         print(" -> ".join(elements) if elements else "Empty list")
105
106
107 if __name__ == "__main__":
108     # Simple usage example for the Singly Linked List.
109
110     # Create an empty linked list.
111     linked_list = LinkedList()
112
113     # Insert elements at the beginning.
114     linked_list.insert_at_beginning(30)
115     linked_list.insert_at_beginning(20)
116     linked_list.insert_at_beginning(10)
117
118     # Insert elements at the end.
119     linked_list.insert_at_end(40)
120     linked_list.insert_at_end(50)
121
122     # Display/traverse the list.
123     # Expected output: 10 -> 20 -> 30 -> 40 -> 50
124     print("Contents of the linked list:")
125     linked_list.display()
126
127
128

```

Output:

```
Contents of the linked list:
10 -> 20 -> 30 -> 40 -> 50
```

Task-4:

Prompt: Implement a Binary Search Tree in Python.

Requirements:

- Node class with value, left, and right.
- Methods: insert() and inorder_traversal().
- Follow BST rules (left < root < right).
- Add comments, docstrings, and example insertions with traversal output.

Code:

```

1 class Node:
2     """
3     A node in a Binary Search Tree (BST).
4
5     Attributes
6     -----
7     value : int | float
8         The value stored in the node.
9     left : Node | None
10        The left child node (stores smaller values).
11     right : Node | None
12        The right child node (stores larger values).
13     """
14
15     def __init__(self, value):
16         """Initialize a new node with the given value."""
17         self.value = value
18         self.left = None
19         self.right = None
20
21
22 class BinarySearchTree:
23     """
24     Simple Binary Search Tree (BST) implementation.
25
26     The BST property is:
27     - All values in the left subtree < node.value
28     - All values in the right subtree > node.value
29     """
30
31     def __init__(self):
32         """Create an empty Binary Search Tree."""
33         self.root = None
34
35     def insert(self, value):
36         """
37         Insert a new value into the BST.
38
39         Parameters
40         -----
41         value : int | float
42             The value to be inserted.
43         """
44         if self.root is None:
45             # Tree is empty; the new node becomes the root.
46             self.root = Node(value)
47         else:
48             # Delegate to the recursive helper starting from the root.
49             self._insert_recursive(self.root, value)
50
51     def _insert_recursive(self, current_node, value):
52         """
53         Recursively insert a value starting from the given node.
54
55         This helper enforces the BST rule: left < root < right.
56         Duplicate values are ignored (no insertion) in this implementation.
57         """
58         if value < current_node.value:
59             # Go to the left subtree.
60             if current_node.left is None:
61                 current_node.left = Node(value)
62             else:
63                 self._insert_recursive(current_node.left, value)
64         elif value > current_node.value:
65             # Go to the right subtree.

```

```

66         if current_node.right is None:
67             current_node.right = Node(value)
68         else:
69             self._insert_recursive(current_node.right, value)
70     else:
71         # Value already exists in the tree; do nothing to avoid duplicates.
72         pass
73
74     def inorder_traversal(self):
75         """
76         Perform an in-order traversal of the BST.
77
78         Returns
79         -----
80         list[int | float]
81             A list of values in ascending (sorted) order.
82         """
83         result = []
84         self._inorder_recursive(self.root, result)
85         return result
86
87     def _inorder_recursive(self, node, result):
88         """
89         Recursive helper to perform in-order traversal.
90
91         Visit left subtree, then current node, then right subtree.
92         """
93         if node is None:
94             return
95         # Traverse left subtree
96         self._inorder_recursive(node.left, result)
97         # Visit current node
98         result.append(node.value)
99         # Traverse right subtree
100        self._inorder_recursive(node.right, result)
101
102
103    if __name__ == "__main__":
104        # Example usage of the BinarySearchTree
105
106        # Create a BST instance
107        bst = BinarySearchTree()
108
109        # Insert some example values
110        values_to_insert = [50, 30, 70, 20, 40, 60, 80]
111        for v in values_to_insert:
112            bst.insert(v)
113
114        # Perform an in-order traversal (should print sorted values)
115        traversal_result = bst.inorder_traversal()
116        print("In-order traversal of BST:", traversal_result)

```

Output:

```

In-order traversal of BST: [20, 30, 40, 50, 60, 70, 80]
In-order traversal of BST: [20, 30, 40, 50, 60, 70, 80]

```

Task-5:

Prompt: Create a Python Hash Table implementation.

Requirements:

- Use an array for storage.
- Handle collisions using chaining (lists).
- Methods: insert(key,value), search(key), delete(key).
- Use hash(key) % table_size.
- Add comments, docstrings, and a small test example.

Code:

```

1 class HashTable:
2     """
3     A simple hash table implementation using:
4     - Python's built-in hash()
5     - Separate chaining with lists to handle collisions
6     """
7
8     def __init__(self, table_size=10):
9         """
10        Initialize the hash table.
11
12        Args:
13            table_size (int): Number of buckets in the underlying array.
14        """
15        self.table_size = table_size
16        # Each bucket will hold a list of (key, value) pairs (for chaining)
17        self.table = [[] for _ in range(table_size)]
18
19    def _hash(self, key):
20        """
21        Compute the array index for a given key using hash(key) % table_size.
22
23        Args:
24            key: The key to hash.
25
26        Returns:
27            int: The index in the table array.
28        """
29        return hash(key) % self.table_size
30
31    def insert(self, key, value):
32        """
33        Insert or update a key-value pair in the hash table.
34
35        If the key already exists, its value is updated.
36
37        Args:
38            key: The key to insert.
39            value: The value associated with the key.
40        """
41        index = self._hash(key)
42        bucket = self.table[index]
43
44        # Check if key already exists in the bucket; if so, update it
45        for i, (k, v) in enumerate(bucket):
46            if k == key:
47                bucket[i] = (key, value)
48                return
49
50        # If key not found, append new (key, value) pair
51        bucket.append((key, value))
52
53    def search(self, key):
54        """
55        Search for a key in the hash table.
56
57        Args:
58            key: The key to search for.
59
60        Returns:
61            The associated value if found, otherwise None.
62        """
63        index = self._hash(key)
64        bucket = self.table[index]
65
66        for k, v in bucket:
67            if k == key:

```

```

68         return v
69
70     return None
71
72     def delete(self, key):
73         """
74         Delete a key-value pair from the hash table if it exists.
75
76         Args:
77             key: The key to delete.
78
79         Returns:
80             bool: True if the key was found and deleted, False otherwise.
81         """
82         index = self._hash(key)
83         bucket = self.table[index]
84
85         for i, (k, v) in enumerate(bucket):
86             if k == key:
87                 # Remove the (key, value) pair from the bucket
88                 del bucket[i]
89                 return True
90
91         return False
92
93
94 if __name__ == "__main__":
95     # Small test example
96
97     # Create a hash table with 5 buckets
98     ht = HashTable(table_size=5)
99
100    # Insert some key-value pairs
101    ht.insert("apple", 10)
102    ht.insert("banana", 20)
103    ht.insert("orange", 30)
104
105    # Update an existing key
106    ht.insert("banana", 25)
107
108    # Search for keys
109    print("apple:", ht.search("apple"))    # Expected: 10
110    print("banana:", ht.search("banana")) # Expected: 25
111    print("orange:", ht.search("orange")) # Expected: 30
112    print("grape:", ht.search("grape"))    # Expected: None (not found)
113
114    # Delete a key
115    print("Delete 'banana':", ht.delete("banana")) # Expected: True
116    print("banana after delete:", ht.search("banana")) # Expected: None
117
118    # Attempt to delete a non-existing key
119    print("Delete 'grape':", ht.delete("grape"))    # Expected: False

```

Output:

```

apple: 10
apple: 10
banana: 25
banana: 25
orange: 30
grape: None
grape: None
Delete 'banana': True
banana after delete: None
Delete 'grape': False
banana after delete: None
Delete 'grape': False
Delete 'grape': False

```